```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler,LabelEncoder
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split,KFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# https://archive.ics.uci.edu/dataset/222/bank+marketing
```

Bank Marketing Dataset:

- The data is related with direct marketing campaigns (phone calls) of a Portuguese banking institution. The classification goal is to predict if the client will subscribe a term deposit (variable y).
- Number of features : **17**
- Number of instances : **45211**
- **FEATURES** -
  - age :
  - job : type of job (categorical: 'admin.','blue-collar','entrepreneur','housemaid','management','retired','self-employed','services','student','technician','unemployed','unknown')
  - marital : marital status (categorical: 'divorced','married','single','unknown'; note: 'divorced' means divorced or widowed)
  - education : categorical: 'basic.4y','basic.6y','basic.9y','high.school','illiterate','professional.course','university.degree','unknown')
  - default : has credit in default?
  - balance : average yearly balance
  - housing : has housing loan?
  - loan : has personal loan?
  - contact : contact communication type (categorical: 'cellular','telephone')
  - day_of_week : last contact day of the week
  - month : last contact month of year (categorical: 'jan', 'feb', 'mar', ..., 'nov', 'dec')
  - duration : last contact duration, in seconds (numeric). Important note: this attribute highly affects the output target (e.g., if duration=0 then y='no'). Yet, the duration is not known before a call is performed. Also, after the end of the call y is obviously known.
  - campaign : number of contacts performed during this campaign and for this client (numeric, includes last contact)
  - pdays : number of days that passed by after the client was last contacted from a previous campaign (numeric; -1 means client was not previously contacted)
  - previous : number of contacts performed before this campaign and for this client
  - poutcome : outcome of the previous marketing campaign (categorical: 'failure','nonexistent','success')
  - y : has the client subscribed a term deposit?

```
banking = pd.read_csv('/content/drive/MyDrive/DM/datasets/bank+marketing/bank/bank-full.csv',sep=";")
banking.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pda
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	

Next steps: Generate code with banking | New interactive sheet

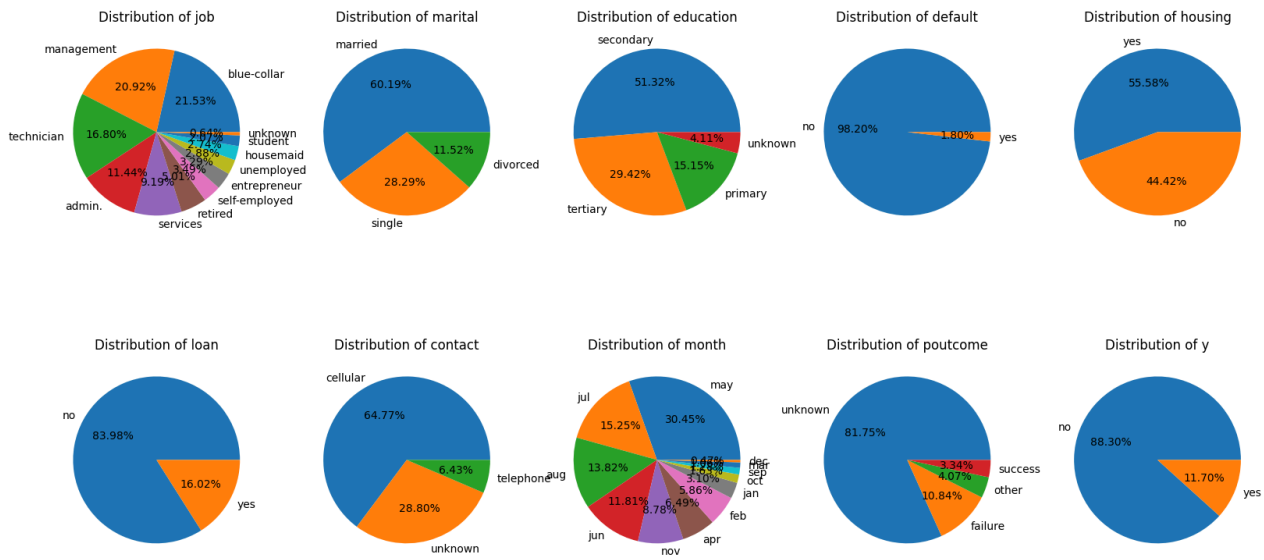
Exploratory Data Analysis

```
banking.info()
```

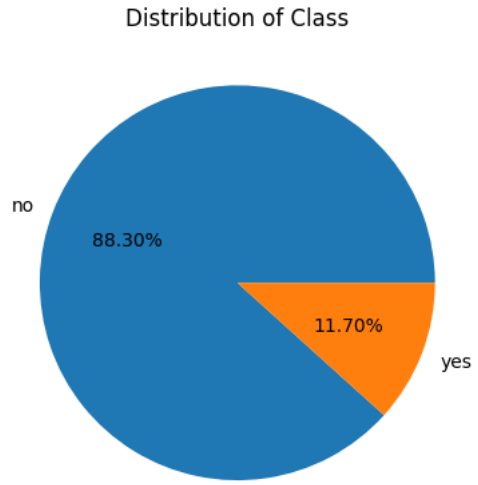
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
```

```
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

```
index = 1
plt.figure(figsize=(20,10))
for col in banking.columns:
  if(banking[col].dtype == 'object'):
    plt.subplot(2,5,index)
    plt.pie(banking[col].value_counts(),labels=banking[col].value_counts().index,autopct="%1.2f%%")
    plt.title(f"Distribution of {col}")
    index+=1
plt.show()
```




```
plt.title("Distribution of Class")
plt.pie(banking['y'].value_counts(),labels=banking['y'].value_counts().index,autopct="%1.2f%%")
plt.show()
```



Preprocessing

```
classes = dict()
def encodeFeatures(col):
  if(col.dtype != 'object'):
    return col

  encoder = LabelEncoder()
  banking[col.name] = encoder.fit_transform(banking[col.name])
  classes[col.name] = encoder.classes_
  return banking[col.name]
```

Encoding features

```
banking = banking.apply(encodeFeatures)
banking.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	pre
0	58	4	1	2	0	2143	1	0	2	5	8	261	1	-1	
1	44	9	2	1	0	29	1	0	2	5	8	151	1	-1	
2	33	2	1	1	0	2	1	1	2	5	8	76	1	-1	
3	47	1	1	3	0	1506	1	0	2	5	8	92	1	-1	
4	33	11	2	3	0	1	0	0	2	5	8	198	1	-1	

Next steps: Generate code with banking New interactive sheet

Checking for any null values

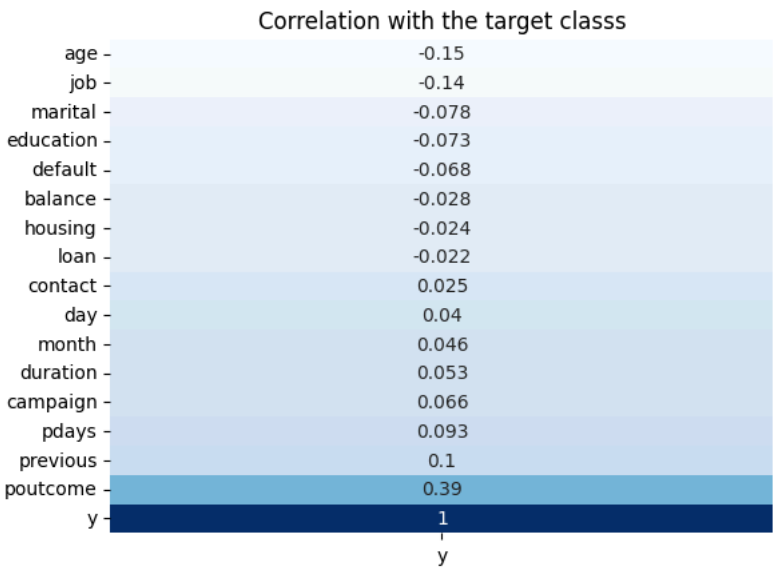
```
banking.isna().any()
```

	0
age	False
job	False
marital	False
education	False
default	False
balance	False
housing	False
loan	False
contact	False
day	False
month	False
duration	False
campaign	False
pdays	False
previous	False
poutcome	False
y	False

dtype: bool

Finding the correlation of the class attribute with each of the feature

```
corr = banking.corrwith(banking['y'])
plt.title("Correlation with the target classs")
sns.heatmap(corr.sort_values().values.reshape(corr.shape[0],1),annot=True,cmap=plt.cm.Blues,cbar=False,xticklabe
plt.show()
```



Separating features from the class

```
X,y = banking.iloc[:,:-1],banking.iloc[:,-1]
X.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	pre
0	58	4	1	2	0	2143	1	0	2	5	8	261	1	-1	
1	44	9	2	1	0	29	1	0	2	5	8	151	1	-1	
2	33	2	1	1	0	2	1	1	2	5	8	76	1	-1	
3	47	1	1	3	0	1506	1	0	2	5	8	92	1	-1	
4	33	11	2	3	0	1	0	0	2	5	8	198	1	-1	

Next steps: Generate code with X New interactive sheet

Applying SMOTE to the class

```
smote = SMOTE()
X,y = smote.fit_resample(X,y)
```

Splitting the dataset into training and test datasets

- training dataset size : 80%
- test dataset size : 20%

```
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=0)
```

Normalization of training dataset

```
scaler = StandardScaler()
X = pd.DataFrame(scaler.fit_transform(X),columns=X.columns)
X.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration
0	1.561707	-0.106938	-0.134959	1.214353	-0.104224	0.189159	1.266024	-0.325192	2.05712	-1.319168	0.935544	-0.338773
1	0.278653	1.561427	1.527436	-0.217344	-0.104224	-0.480422	1.266024	-0.325192	2.05712	-1.319168	0.935544	-0.654264
2	-0.729461	-0.774284	-0.134959	-0.217344	-0.104224	-0.488974	1.266024	3.075103	2.05712	-1.319168	0.935544	-0.869372
3	0.553593	-1.107957	-0.134959	2.646050	-0.104224	-0.012602	1.266024	-0.325192	2.05712	-1.319168	0.935544	-0.823482
4	-0.729461	2.228773	1.527436	2.646050	-0.104224	-0.489291	-0.789874	-0.325192	2.05712	-1.319168	0.935544	-0.519463

Next steps: Generate code with X New interactive sheet

```
y_train.head()
```

	y
44364	0
56443	1
17870	1
67004	1
45233	1

dtype: int64

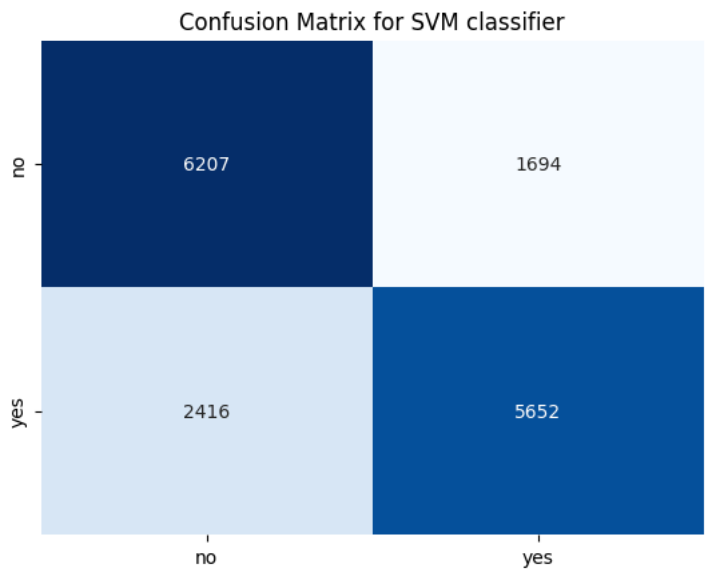
Applying SVM model

```
svm = SVC()
svm_classifier = svm.fit(x_train,y_train)
y_pred_svm = svm_classifier.predict(x_test)
y_pred_svm
```

```
array([1, 0, 0, ..., 0, 1, 0])
```

Confusion Matrix for SVM

```
svm_conf_matrix = confusion_matrix(y_test,y_pred_svm)
plt.title("Confusion Matrix for SVM classifier")
sns.heatmap(svm_conf_matrix,annot=True,cmap=plt.cm.Blues,cbar=False,fmt="d",xticklabels=classes['y'],yticklabels
plt.show()
```



Performance of SVM

```
report_svm = classification_report(y_test,y_pred_svm,output_dict=True)
report_svm = pd.DataFrame(report_svm)
print(accuracy_score(y_test,y_pred_svm))
report_svm
```

```
0.7426263385309037
```

	0	1	accuracy	macro avg	weighted avg
precision	0.719819	0.769398	0.742626	0.744609	0.744868
recall	0.785597	0.700545	0.742626	0.743071	0.742626
f1-score	0.751271	0.733359	0.742626	0.742315	0.742221
support	7901.000000	8068.000000	0.742626	15969.000000	15969.000000

Next steps: Generate code with `report_svm` New interactive sheet

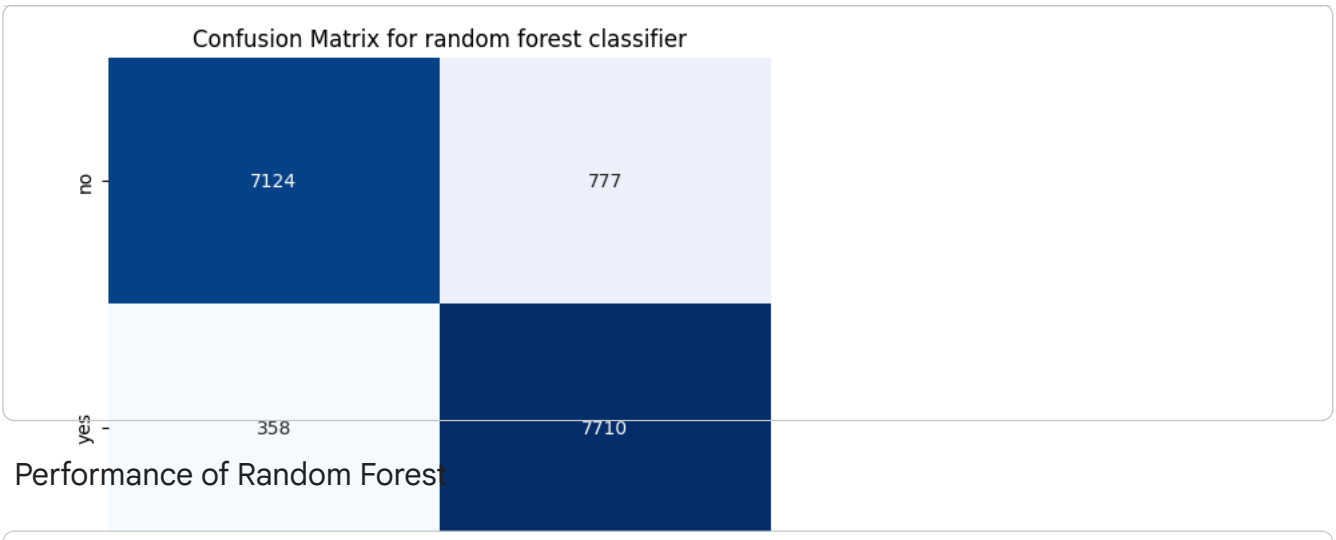
Applying Random Forest

```
forest = RandomForestClassifier()
forest_classifier = forest.fit(x_train,y_train)
y_pred_forest = forest_classifier.predict(x_test)
y_pred_forest
```

```
array([1, 1, 1, ..., 1, 0, 1])
```

Confusion Matrix for Random Forest

```
svm_conf_matrix = confusion_matrix(y_test,y_pred_forest)
plt.title("Confusion Matrix for random forest classifier")
sns.heatmap(svm_conf_matrix,annot=True,cmap=plt.cm.Blues,cbar=False,fmt="d",xticklabels=classes['y'],yticklabels
plt.show()
```



Performance of Random Forest

```
report_forest = classification_report(y_test,y_pred_forest,output_dict=True)
report_forest = pd.DataFrame(report_forest)
print(accuracy_score(y_test,y_pred_forest))
report_forest
```

```
0.9289247917840816
```

	0	1	accuracy	macro avg	weighted avg
precision	0.952152	0.908448	0.928925	0.930300	0.930072
recall	0.901658	0.955627	0.928925	0.928643	0.928925
f1-score	0.926217	0.931441	0.928925	0.928829	0.928856
support	7901.000000	8068.000000	0.928925	15969.000000	15969.000000

Next steps: Generate code with report_forest New interactive sheet